

AI Native Agent 产品闭环与自进化手册

从任务完成到受控进化的产品方法论

系列海报

01 / 09

总览与问题定义

任务完成 Agent 执行 Trace 观测 Eval 评估 反馈迭代

1 这套讨论到底在研究什么？

不是研究



- Prompt 技巧
- 聊天机器人
- 多 Agent 炫技
- 任意自我修改

VS

而是研究



- 任务闭环
- 可靠执行
- 过程观测
- 质量评估
- 反馈迭代
- 受控进化

2 一句话定义

“AI Native Agent 产品不是“传统软件 + AI 聊天框”，而是围绕任务完成、过程观测、质量评估、真实反馈和受控迭代构建的产品系统。

核心从“回答问题”转向“完成任务”，从“一次输出”转向“持续变好”。

3 产品公式

AI Native Agent 产品

- 🎯 = 任务场景
- 🤖 + Agent 执行链路
- 📈 + Trace 可观测性
- 🛡️ + Agent Eval 体系
- 🔄 + Feedback 数据闭环
- 🔄 + Controlled Iteration
- 🛡️ + Governance

4 核心变化：从生成答案到完成任务

1 目标变了

回答问题
→ 完成任务

2 逻辑变了

规则流程
→ 模型推理 +
工具调用

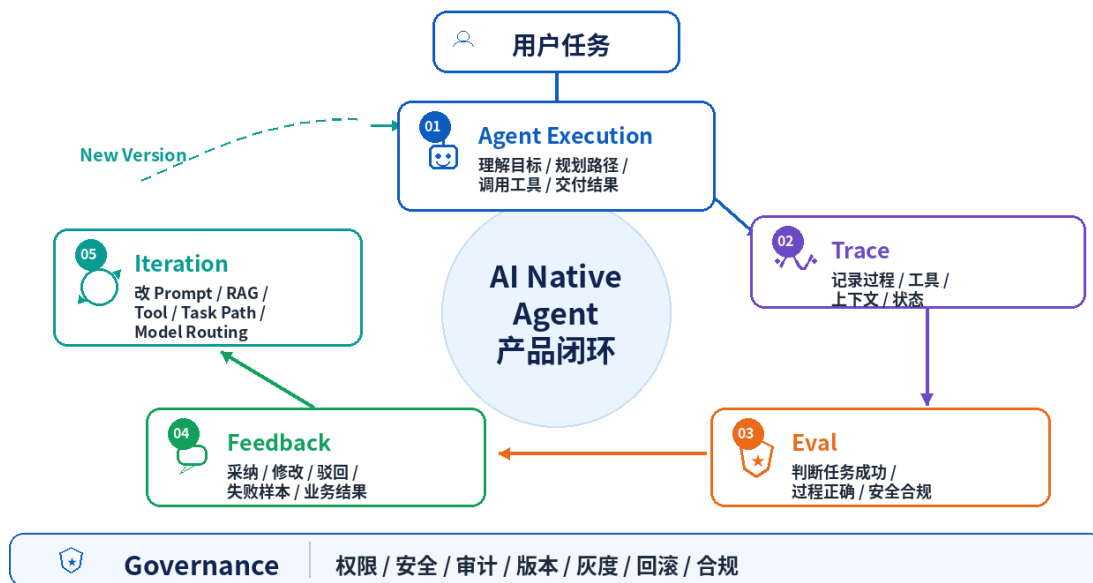
3 质量变了

答案正确
→ 任务成功 +
过程安全

4 迭代变了

需求上线
→ 失败样本 +
评测反馈驱动

5 五层闭环：AI Native Agent 产品如何持续变好？



6 9 张图讲清 AI Native Agent 产品闭环



AI Native 的本质，不是让 AI 进入界面，而是让产品具备任务闭环。

任务完成不是终点。可观测、可评估、可反馈、可迭代，才是 AI Native 产品真正的闭环。

第一性原理：从信息交互到任务完成

为什么 AI Native 不是传统 SaaS 加聊天框？

系列海报

02 / 09

第一性原理

任务完成

范式迁移

闭环驱动

质量指标

1 核心判断

AI Native Agent 产品的第一性原理不是“生成答案”，而是：

围绕任务完成，建立可观测、可评估、可反馈、可迭代、可治理的持续进化闭环。

2 范式迁移：产品目标变了



3 AI Native vs 传统 SaaS

维度	传统 SaaS / App	AI Native Agent 产品
输入	表单 / 按钮 / 固定流程	自然语言 / 多模态 / 上下文意图
逻辑	规则和代码驱动	模型推理 + 工具调用 + 流程路径
输出	页面 / 数据 / 报表	建议 / 动作 / 解释 / 任务结果
质量	测试 / 监控 / QA	Trace / Eval / 人审 / 回归评测
迭代	需求开发上线	失败样本 + 评测结果 + 反馈数据

4 核心指标也变了



5 最小 AI Native 产品闭环



如果没有“反馈记录 → 失败样本 → 评估 → 迭代”，它更像 AI 功能，而不是完整 AI Native 产品系统。

系列导航



AI Native 不是把 AI 放进产品，而是用 AI 重构产品如何理解、执行和迭代。

Agent 执行链路：从目标到结果

Agent 如何在有限上下文、权限和预算下可靠完成任务？

系列海报

03 / 09

Agent 执行链路

Agent Loop

Harness

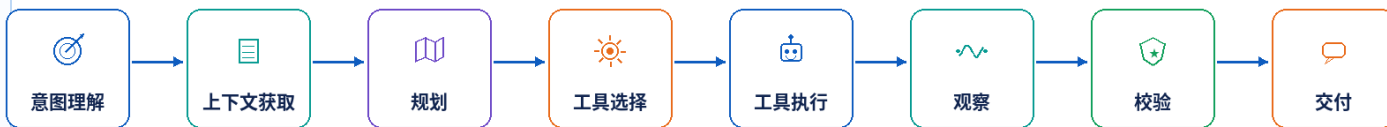
Tool Calling

任务路径

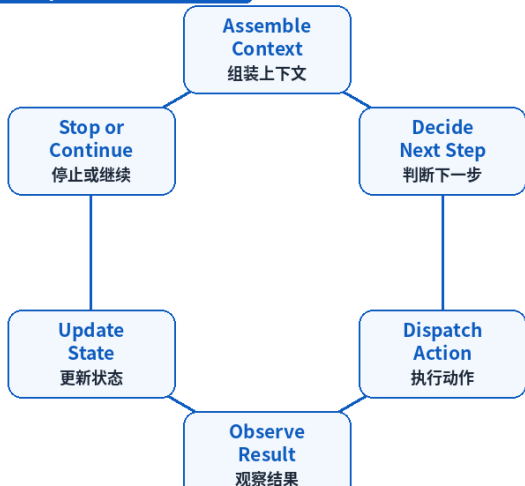
1 一句话结论

Agent 执行能力不是“模型更聪明”，而是把用户目标转化为可规划、可调用工具、可观察、可校验、可交付的任务执行链。

2 线性执行链路



3 Agent Loop：受控运行循环



没有停止条件的 Agent Loop，不是智能，而是不受控。

4 Agent Harness：工程外壳

生产级 Agent 不只是模型：

+ LLM

+ Agent Loop

+ Tool Runtime

+ Context / Memory

+ Permissions / Sandbox

+ Trace / Eval

+ Verification / Recovery

+ Human Control

Loop 是内核，Harness 是让它可靠、可控、可恢复的外壳。

5 任务路径控制权分层

层级	名称	路径控制权
L1	知识问答型 Assistant	用户 / 单轮交互
L2	工具调用型 Assistant	用户意图 + 短链路模型决策
L3	流程受控型 Agent	系统定义主流程，Agent 做局部智能
L4	自主规划型 Agent	Agent 动态规划路径
L5	长周期自治 Agent	环境触发 + Agent 持续运行

L3 / L4 的关键区别

L3：系统替 Agent 规划路，Agent 走好每一步。

L4：Agent 自己规划路，系统负责画边界。

系列导航



Agent 的价值不在于像人一样聊天，而在于像系统一样完成任务。

Trace 与可观测性：让执行过程可见

没有 Trace，就没有真正的评估、调试和治理。

系列海报

04 / 09

Trace 与可观测性

1 一句话定义

Trace 是一次端到端任务执行轨迹，记录 Agent 如何从用户目标一步步走到最终结果。

2 基础术语

概念	简明解释
Trace	一次任务的完整故事
Span	Trace 中的一个步骤
Event	关键状态变化
Log	某个时间点的细节记录
Metric	跨任务统计出的指标

3 Agent Trace 记录什么？

User Task

用户目标

System Context

规则 / 权限 / 版本

Model Calls

模型输入输出摘要

Retrieved Context

检索内容

Tool Calls

工具 / 参数 / 结果

State Changes

计划 / 步骤 / 预算

Guardrail Events

拦截 / 脱敏 / 风险标记

Final Outcome

成功 / 失败 / 采纳 / 人工

4 Trace ≠ Log

Log

告诉你某个点
发生了什么

Trace

告诉你一次任务
如何走到结果

5 Trace 是连接层

Agent Execution

Eval

Trace
事实记录

Feedback

Governance

6 可观测粒度：从任务到风险



任务级

是否完成任务



步骤级

每一步是否合理



工具级

调用是否正确



上下文级

依据是否可靠



风险级

是否越权或不安全

系列导航

01



总览

02



原理

03



执行

04



Trace

05



Eval

06



Feedback

07



Iteration

08



边界

09



地图



没有 Trace，就只能评价答案；有了 Trace，才能评价任务。

Agent Eval 体系：从答案准确率到任务成功率

Eval 是 AI Native Agent 产品迭代的度量尺。

系列海报

05 / 09

Agent Eval 体系

- Outcome Eval
- Trace Eval
- Grader
- Regression Set

1 一句话定义

Agent Eval 判断的是 Agent 是否可靠完成任务，而不是最终回答看起来对不对。

2 从 LLM Eval 到 Agent Eval

维度	普通 LLM Eval	Agent Eval
对象	Prompt → Response	Task → Multi-step Execution → Outcome
重点	答案质量 / 格式 / 事实性	任务成功 / 过程 / 工具 / 安全 / 成本
数据	测试集 / 问答样本	Trace / 工具结果 / 用户反馈 / 人审
失败	答错 / 幻觉	上下文错 / 工具错 / 越权 / 空转

3 两类核心评估



Outcome Eval

最终结果或环境状态是否达标



Trace Eval

执行过程是否正确
为什么成功或失败

4 Agent Eval 的关键维度

- Task Success**
任务是否完成
- Outcome Correctness**
结果是否正确
- Tool Use**
工具调用是否正确
- Context Quality**
上下文是否可靠
- Process Quality**
步骤是否合理
- Safety & Policy**
安全与权限
- Cost & Latency**
成本与延迟
- User Value**
用户价值

5 Eval 如何进入迭代闭环？



系列导览

- 01
总览
- 02
原理
- 03
执行
- 04
Trace
- 05
Eval
- 06
Feedback
- 07
Iteration
- 08
边界
- 09
地图



AI Native 产品不能只问“答得对不对”，还要问“事有没有办成”。

Feedback 数据闭环：从反馈信号到迭代资产

Feedback 不是点赞点踩，而是真实世界信号进入改进系统。

- User Feedback
- Human Review
- Business Outcome
- Failure Case

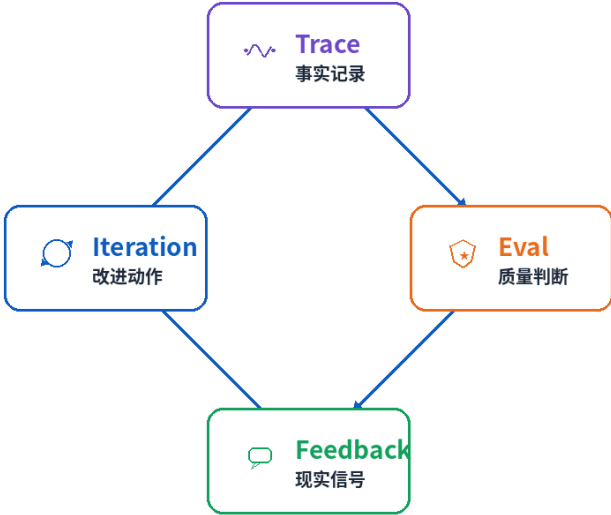
1 一句话定义

Feedback 是把用户反馈、人工审核、线上 Trace、业务结果和失败样本转化为 Agent 改进燃料的数据闭环。

2 Feedback 来源



3 四个核心对象的关系



4 从线上反馈到评测集



5 失败样本资产化



系列导览



反馈的价值不在于知道用户不满意，而在于把不满意转成下一轮改进数据。

受控迭代机制：让产品进化可管理

拿到 Trace、Eval、Feedback 之后，下一版到底改什么？

系列海报

07 / 09

受控迭代机制

Controlled Change

Release Gate

Canary

Rollback

1 一句话定义

受控迭代是基于证据、评测、版本、灰度和回滚的 Agent 系统改进流程。

2 迭代对象全景：不只是 Prompt



3 受控迭代流程



4 根因不要错配

现象	更可能的根因	优先改哪里
检索不到资料	上下文不足或索引不佳	Context / RAG
工具参数错	工具描述或参数结构不清	Tool Schema
Agent 空转	缺少状态、预算或停止条件	Loop Control / State
高风险动作直接执行	缺少权限门禁	Guardrails / HITL
分数高但用户不满意	评测集或指标不贴近真实任务	Dataset / Metrics

系列导览



Agent 迭代更像行为系统的迭代，必须靠 Eval 和 Trace 管住不确定性。

自进化边界：什么可以自动化，什么必须受控？

自进化不是放任 Agent 自己改自己，而是在治理框架内提高自动化程度。

Assisted Improvement

Human Approval

Guardrails

Governance

系列海报

08 / 09

自进化边界

1 一句话定义

自进化 = 系统基于 Trace、Eval、Feedback 自动发现问题、提出改进、验证候选，并在治理边界内逐步应用。

2 自进化三级：自动化程度不同，风险不同

1

Assisted Improvement

AI 帮人发现问题、提出建议

低风险

2

Semi-Automated Optimization

AI 生成候选改动，需 Eval 和审批

中风险

3

Autonomous Self-Modification

AI 自动修改并上线

高风险

3 边界设计：可以自动化 / 必须确认 / 禁止自动化

可以自动化

- ✓ 失败样本聚类
- ✓ Trace 摘要
- ✓ 根因候选分析
- ✓ Prompt 改进候选
- ✓ Eval case 生成
- ✓ RAG / rerank 候选
- ✓ Model routing 候选

必须人工确认

- 系统级 Prompt
- 工具权限
- 新增写操作工具
- 放宽 Guardrails
- 生产发布
- 高风险领域策略
- 修改 Grader / Rubric

禁止自动化

- ✗ 自行提升权限
- ✗ 自行关闭护栏
- ✗ 删除失败日志或 Trace
- ✗ 修改评分标准并声称变好
- ✗ 未审核使用敏感数据
- ✗ 高风险不可逆动作

4 受控自进化路径



系列导航

01



总览

02



原理

03



执行

04



Trace

05



Eval

06



Feedback

07



Iteration

08



边界

09



地图



AI Native 产品可以越来越聪明，但不能越来越失控。

知识体系图：把产品闭环压缩成一张地图

从任务完成到受控进化，形成可观测、可评估、可治理的产品系统。

系列海报

09 / 09

知识体系图

Knowledge Map

Product Loop

Governance

Practice

1 中心命题

AI Native Agent 产品 = 任务完成 + 持续进化 + 治理边界

2 总知识地图



3 主闭环：从任务到新版本



每一轮迭代都必须回到 Agent Execution，并形成可验证的新版本。

4 横切治理层



Governance 不是最后一步，而是贯穿执行、观测、评估和迭代的底座。

5 从知识体系到项目实践



选择真实任务场景



设计 Agent Harness



建立 Trace / Eval 基线



用 Feedback 推动受控迭代

系列导航

01



总览

02



原理

03



执行

04



Trace

05



Eval

06



Feedback

07



Iteration

08



边界

09



地图



AI Native Agent 产品的终局，不是更像聊天机器人，而是更像一个能持续学习的任务系统。